in section 11.3. Rather than giving clients direct access to the private key to create their own JWTs, a separate controller process generates JWTs on their behalf and distributes these short-lived bearer tokens to the pods that need them. The client then uses the JWT bearer grant type to exchange the JWT for a longer-lived access token (and optionally a refresh token too). In this way the JWT bearer grant type can be used to enforce a separation of duties that allows the private key to be kept securely away from pods that service user requests. When combined with certificate-bound access tokens of section 11.4.6, this pattern can result in significantly increased security for OAuth2-based microservices.

The main problem with short-lived credentials is that Kubernetes is designed for highly dynamic environments in which pods come and go, and new service instances can be created to respond to increased load. The solution is to have a controller process register with the Kubernetes API server and watches for new pods being created. The controller process can then create a new temporary credential, such as a fresh signed JWT, and deploy it to the pod before it starts up. The controller process has access to long-lived credentials but can be deployed in a separate namespace with strict network policies to reduce the risk of it being compromised, as shown in figure 11.8.